IterGen на SQL: краткий отчёт

1. Задача и идея метода

Я сравнил обычную генерацию, SynCode и свой цикл IterGen на модели Qwen3.5-0.8B. SynCode ограничивает форму SQL с помощью грамматики. IterGen позволяет дополнительно проверить завершённую часть запроса и удалить её, если проверка не пройдена. Грамматически допустимое имя при этом может отсутствовать в конкретной базе.

При локальном откате сохраняется предыдущая часть SQL. Например, после удаления ошибочной таблицы из `SELECT id FROM db_info` можно продолжить после `SELECT id FROM`. Полная перегенерация заново строила бы весь ответ. Движение по символам грамматики и проверка имён взяты из идеи IterGen.

2. Выборка и условия

Я зафиксировал seed 228 и выбрал по 10 задач из пяти баз Spider: flight_2, wta_1, world_1, student_transcripts_tracking и network_1. Ещё по одной задаче из каждой базы оставил для отладки; пересечений с основными 50 нет. Задачи и промпты одинаковы для всех режимов. Это выборка с одинаковым вкладом баз, а не полное распределение Spider.

Модель — Qwen/Qwen3.5-0.8B, видеокарта — RTX 3060, dtype — float16. Использовались PyTorch 2.14.1+cu130 и Transformers 5.3.0, `do_sample=False`, предел ответа 200 токенов. Seed библиотек и детерминированные алгоритмы включены. На одном отладочном примере повторные ответы трёх режимов совпали; полный повтор 50 задач не выполнялся.

3. Реализация и патчи

Из авторского кода взяты SQL-грамматика, Dataset Spider, обработка ответа и методы start / forward / view / backward. Самостоятельно написаны цикл проверок, правила глубокого отката, сохранение результатов и оценка SQL. Происхождение кода отмечено и в ноутбуке.

В IterGen я добавил передачу уже загруженной модели, токенизатора и масок, настройки совместимости с Transformers и кэш Qwen3.5. После отката этот кэш создаётся заново: модель пересчитывает сохранённый префикс. Также исправлена остановка до первого токена из-за стоп-строки в промпте. В SynCode маски преобразуются по одному состоянию, одинаковые маски переиспользуются. Кэш сохраняется через временный файл; ошибка чтения больше не запускает незаметное перестроение.

В статье оценивались 1034 задачи и другие модели. Здесь воспроизведена идея на заданной небольшой модели, с собственными бюджетами и оценщиком. Оптимизация KV-кэша статьи для отката Qwen3.5 не воспроизведена.

4. Проверки и ограничения цикла

Я проверяю каждое завершённое имя таблицы или столбца. Раньше проверять нельзя: часть `high` ещё может превратиться в `highschooler`. Если ждать конца SQL, модель успеет написать лишнее продолжение после ошибки. Полное выполнение запроса используется при итоговой оценке.

При ошибке выполняется локальный backward. После двух локальных неудач в том же месте третья может дополнительно вызвать откат до последнего select_expr, сохраняя SELECT. Пределы: 25 вызовов forward, 8 откатов, 200 токенов сохранённого ответа и общий бюджет 300 токенов. Последний проверяется между forward, поэтому может быть превышен внутри последнего шага.

Авторская история веток и `recurrence_penalty=0.3` меняют оценки уже пробованных токенов. Это важно при генерации без случайности. Но в данной версии коэффициент умножает логиты и не всегда понижает отрицательную оценку. Строгого запрета повторной ветки здесь нет.

5. Метрики и результаты

Выполнимость — доля SQL без ошибки SQLite. Execution accuracy — доля совпадений результатов с эталоном. Порядок учитывается при ORDER BY в эталоне; иначе сравниваются строки с количеством повторов. Совпадение пустых результатов тоже засчитывается. Это упрощённый оценщик, не официальный Spider evaluator.

Режим	Без ошибки	Совпал результат	Время, с	Токены	Откаты
Обычная генерация	28/50 (56%)	14/50 (28%)	148.2	2543	0
SynCode	12/50 (24%)	7/50 (14%)	239.0	4535	0
IterGen, 8 + глубокий	28/50 (56%)	14/50 (28%)	167.1	2514	18
IterGen, 2 отката	27/50 (54%)	13/50 (26%)	159.6	2362	16
IterGen, 8 локальных	28/50 (56%)	14/50 (28%)	165.6	2514	18

Время не включает загрузку модели, DFA и итоговую проверку SQL; у IterGen включает чтение схемы. Учтены и удалённые токены. Основной IterGen дал ту же accuracy, что обычная генерация, при дополнительном времени около 12.8%. Средние затраты: 3.34 с, 50.28 токена и 0.36 отката на задачу. Откаты были в 12 задачах; три из них дали совпадение.

6. Что показали абляции

При лимите 2 вместо 8 потеряно одно совпадение — ID 872. Оба варианта сделали два отката, но ограниченный сразу остановился с `SELECT id FROM`. Основной смог вызвать следующий forward. Значит, здесь помогло правило продолжения после второго отката, а не третья попытка отката.

Глубокий откат на основных 50 задачах не использовался. Без него SQL, трассы и токены совпали, поэтому его польза не измерена. Имена можно исправить, но дополнительные попытки не устраняют смысловую ошибку, которую наша проверка не замечает. Параметры по eval не подбирались. Бонус с полной перегенерацией при равном бюджете не реализован: отделить пользу отката от дополнительных попыток пока нельзя.

7. Четыре примера с фрагментами трасс

ID 872, network_1, eval — исправление имён. На шаге 1 получен `SELECT db_id FROM`: столбец db_id не найден, backward оставил `SELECT`. На шаге 3 получен `SELECT id FROM db_info WHERE`: таблица db_info не найдена, осталось `SELECT id FROM`. Итог: `SELECT id FROM highschooler WHERE name = 'Kyle';`. Результат совпал с эталоном после двух откатов.

ID 456, wta_1, debug — откат помог локально, но ответ неверный. На шагах 12 и 13 проверка отвергла db_id и wta_1 после `WHERE m.loser_id =`. Оба раза backward удалял имя. Итоговое условие: `WHERE m.loser_id = p.player_id ORDER BY p.birth_date;`. SQL выполнился, но лишние JOIN и столбец birth_date изменили результат относительно эталона с first_name и last_name.

ID 580, student_transcripts_tracking, debug — пропущенный алиас. На шаге 1 в `SELECT DISTINCT s.first_name,` проверка имени прошла. Итоговый фрагмент — `FROM students ORDER BY s.other_student_details ASC;`, но алиас s не объявлен. SQLite сообщил `no such column: s.first_name`; откатов не было. Проверка ищет столбцы по всей базе и не разрешает алиасы. В эталоне также нужна сортировка DESC.

ID 754, world_1, debug — совпадение пустых результатов. На шаге 1 `SELECT c.code,` прошло проверку c.code, действие continue. Итоговый SQL содержит `WHERE cl.isofficial =1 OR cl.isofficial =0;`, а эталон ищет страны с официальным английским или нидерландским языком. Оба запроса вернули ноль строк: метрика засчитала совпадение, хотя смысл SQL отличается. Откатов не было.

Полные вопросы, эталоны, SQL и трассы вынесены в TRACE_EXAMPLES.md. Отдельный пример ID 1029 с глубокими откатами и исчерпанием бюджета описан там же и не включён в общую метрику; отдельного JSONL для него сейчас нет. Статус stopped означает завершение генерации, а не правильность ответа.

8. Выводы и перенос на агентов

На этой выборке IterGen не улучшил общую accuracy. Он исправил три задачи относительно обычной генерации и потерял три другие; у потерянных откатов не было. Для вывода о скорости и качестве нужны другие выборки и повторы. Маски SynCode также могли использовать более широкую замену при предупреждении Exact lookup not found. Проверка схемы не оценивает нужные фильтры, JOIN и агрегаты.

Для агента можно снаружи проверять форму вызова, наличие инструмента, аргументы, права и бюджет до выполнения. Если допустимость зависит от состояния, например статуса заказа, его нужно проверить непосредственно перед действием. При параллельных изменениях проверку и действие нужно согласовать. Откат текста не отменяет отправленное письмо или платёж: нужны отдельные отмена, компенсация и защита от повторов. Наш SQL-эксперимент не проверяет меняющееся состояние, внешние действия и цепочки инструментов.

9. Запуск и артефакты

Из корня проекта открыть Test_task_RP.ipynb с ядром .venv/bin/python, проверить cache_dir и SQLite-базы, затем выполнить ячейки по порядку. Для нового скачивания баз нужно заполнить SPIDER_URL; сейчас там пробел. После правок библиотеки ядро перезапустить. Зависимости указаны в itergen/requirements.txt; версии запуска — в run_config.json.

Выборка, предсказания и трассы лежат в results/seed_228/. При смене кода нужна отдельная папка прогона: конфиг не содержит хешей исходников. Обычные тесты цикла и оценки: `.venv/bin/python -m unittest discover -s tests -v`.